



UNIVERSITY OF
WEST LONDON
The *Career* University

**Integrated Database and Analytics Solution
NorthStar Urban Mobility and Logistics**

**Databases and Analytics (CS) (09-FEB-26 - 22-MAY-26)
CP6CS56E_A_SEM2_202526**

Name: Kundan Kumar Ray Yadav

**Student ID: 33116717
Course: BSc Computer Science**

Group: Tuesday

Module Leader: Shidrokh Goudarzi

Submission Date: 07/05/2026

GitHub Repository: <https://github.com/KundanVK18/northstar-database-analytics-assignment>

Executive Summary

This report develops an integrated database and analytics solution for NorthStar Urban Mobility and Logistics. The case study describes a fast-growing organisation that has combined transport, last-mile delivery, warehousing, EV charging and digital platform services, but still relies on fragmented data. The business problem is therefore not simply poor delivery performance. It is a data architecture problem that prevents managers from seeing how orders, deliveries, complaints, incidents, vehicles, hubs and app events interact.

The supplied dataset contains 1,250 orders, 950 delivery records, 650 customers, 170 drivers, 120 vehicles, 320 complaints, 280 incidents, 640 app events and 8 hubs. The analysis found 132 failed deliveries (13.89%) and 202 delayed deliveries (21.26%). The Central zone has the highest failure rate at 20.16%, route override cases are riskier than non-override cases (15.61% compared with 11.53%), and vehicles marked InRepair have a much higher failure rate of 30.31%.

Customer evidence also highlights specific pressure points. Passenger services generated the largest number of complaints (84), while Business services had the highest failure rate by service type (19.84%). In total, 610 of 950 delivery cases (64.21%) met at least one risk condition, such as failure, delay, complaint, incident, low rating or compensation. These results support a hybrid solution: retain structured data for SQL, R and Python analytics, while using MongoDB Atlas for nested service-case records that combine complaints, incidents, route exceptions and app events.

Table of Contents

1. Introduction.....	4
2. Case Study Problem Analysis	4
3. Dataset Description and Preparation.....	5
Relationship model.....	5
Data preparation steps	5
4. SQL in R Analysis	6
RSQLite setup	6
Query 1: failure rate by zone.....	6
Query 2: complaints by service type.....	6
5. R Analytics and Visualisation	7
6. Python Data Processing and Visualisation.....	9
7. MongoDB Atlas NoSQL Database Design.....	11
Example service_cases document	11
8. CRUD Operations Using PyMongo	11
9. MongoDB Aggregation Queries	12
Aggregation examples	12
10. Indexing and Query Optimisation	12
11. Key Findings and Business Recommendations	13
Recommended target architecture	13
12. Critical Evaluation and Limitations	14
13. Conclusion	14
14. References.....	14
15. Appendix: GitHub and Colab Evidence	15

1. Introduction

NorthStar Urban Mobility and Logistics operate across several UK cities and manages passenger services, last-mile delivery, warehouse dispatch, electric vehicle charging hubs, route planning and mobile customer services. The company has enough data to understand its operations, but the case study shows that the data is spread across separate systems and is not being turned into reliable insight (University of West London, 2026b).

The main issue is that older relational records and newer semi-structured platform records do not work together effectively. Structured systems record bookings, deliveries, payments, driver assignments and vehicle information. Newer systems produce app events, status changes, route overrides, exception records and complaint histories. When these sources are analysed separately, managers only see symptoms: late deliveries, rising complaints or high costs. They do not see the chain of evidence behind those outcomes.

The purpose of this report is to design and demonstrate an integrated solution using SQL in R, R analytics, Python data processing, MongoDB Atlas and query optimisation. The approach is deliberately practical. It uses SQL and R-style analysis for structured records, Python for data preparation and repeatable analytics, and MongoDB for flexible service-case documents that store nested event histories. The coursework brief requires these technologies to be applied as one coherent solution rather than as isolated exercises (University of West London, 2026a).

2. Case Study Problem Analysis

NorthStar's issues are interconnected. A poor customer experience may begin with a high-priority order, continue through route allocation, involve a driver override or vehicle fault, and end as a complaint or compensation cost. A distinction-level response therefore needs to connect operational evidence rather than only describe each file in the dataset.

Problem area	Why it matters	Evidence required
Fragmented systems	Prevents a reliable single view of customer and operational performance	Joined order, delivery, complaint, incident, vehicle and app-event records
Service failure and delay	Directly damages customer trust and service-level performance	Failure and delay rates by zone, hub, service type and vehicle
Complaint disconnect	Complaints may not match the status recorded in operational systems	Complaint type, status, compensation and linked order/delivery outcomes
Manual route overrides	May show poor routing, local constraints or driver behaviour issues	Override counts compared with failures, cost, distance and ratings
Vehicle downtime	Late maintenance detection can increase failures and operating cost	Maintenance status, battery health, incidents and delivery outcomes
Financial opacity	Cost, exception and service data are stored separately	Average cost, compensation, ratings, failures and service type

The analysis focuses on six questions: which zones and hubs have the highest failure rates, which services generate complaints, whether route overrides are linked to weaker outcomes, whether vehicle condition affects service reliability, which app events show reliability problems, and which service cases should be treated as high risk.

3. Dataset Description and Preparation

The dataset is appropriate because it covers both demand and execution: customers and orders show what was requested; deliveries, drivers, vehicles and hubs show how the service was performed; complaints, incidents and app events show the customer and operational consequences.

Dataset	Records	Role in the analysis
hubs.csv	8	Hub and zone context
customers.csv	650	Customer profile and engagement data
drivers.csv	170	Driver experience, training and rating
vehicles.csv	120	Fleet condition and maintenance status
orders.csv	1,250	Service demand, zones, value and priority
deliveries.csv	950	Delivery outcome, route, rating and cost
incidents.csv	280	Operational incidents linked to deliveries
complaints.csv	320	Customer dissatisfaction and compensation
app_events.csv	640	Mobile-platform activity and reliability

Relationship model

```
customers.customer_id -> orders.customer_id -> deliveries.order_id -> incidents.delivery_id
orders.order_id -> complaints.order_id
orders.order_id -> app_events.order_id
deliveries.driver_id -> drivers.driver_id
deliveries.vehicle_id -> vehicles.vehicle_id
deliveries.hub_id -> hubs.hub_id
```

Data preparation steps

- Converted date fields to datetime format for time-based analysis.
- Standardised zone names to avoid duplicate categories such as AIRPORT and Airport.
- Created flags for failed deliveries, delayed deliveries, route overrides, app failures and high-risk cases.
- Summarised complaints, incidents and app events before joining them into the main analytical dataset.
- Created an integrated delivery-level dataset containing operational, customer, driver, vehicle, hub and service-quality fields.

```
analysis_df = (deliveries
    .merge(orders, on='order_id', how='left')
    .merge(customers, on='customer_id', how='left')
    .merge(drivers, on='driver_id', how='left')
    .merge(vehicles, on='vehicle_id', how='left')
    .merge(hubs, on='hub_id', how='left'))
```

4. SQL in R Analysis

SQL is used because the core operational data is relational in structure. The same SQL logic can be run through R using DBI and RSQLite, which supports the assignment requirement for SQL in R while also allowing repeatable queries and clear outputs.

RSQLite setup

```
library(DBI)
library(RSQLite)
library(dplyr)
con <- dbConnect(RSQLite::SQLite(), 'memory:')
dbWriteTable(con, 'orders', orders, overwrite = TRUE)
dbWriteTable(con, 'deliveries', deliveries, overwrite = TRUE)
dbWriteTable(con, 'hubs', hubs, overwrite = TRUE)
dbWriteTable(con, 'complaints', complaints, overwrite = TRUE)
```

Query 1: failure rate by zone

```
SELECT h.zone,
       COUNT(d.delivery_id) AS deliveries,
       SUM(CASE WHEN LOWER(d.delivery_status) = 'failed' THEN 1 ELSE 0 END) AS failed,
       ROUND(100.0 * SUM(CASE WHEN LOWER(d.delivery_status) = 'failed' THEN 1 ELSE 0 END)
            / COUNT(d.delivery_id), 2) AS failure_percent
FROM deliveries d
JOIN hubs h ON d.hub_id = h.hub_id
GROUP BY h.zone
ORDER BY failure_percent DESC;
```

Zone	Deliveries	Failed	Failure %	Delayed %	Avg rating
Central	243	49	20.16%	19.34%	3.78
Airport	104	15	14.42%	25.96%	3.88
West	127	16	12.60%	22.05%	3.92
North	136	17	12.50%	19.12%	3.84
Riverside	115	14	12.17%	21.74%	3.88
South	106	10	9.43%	24.53%	3.95
East	119	11	9.24%	19.33%	3.90

Interpretation: Central is the clearest operational hotspot, with 49 failed deliveries and a 20.16% failure rate. This is more informative than counting failures alone because the percentage adjusts for different delivery volumes across zones.

Query 2: complaints by service type

```
SELECT o.service_type,
       COUNT(c.complaint_id) AS complaints,
       ROUND(AVG(c.resolution_days), 2) AS avg_resolution_days,
```

```

ROUND(SUM(c.compensation_amount), 2) AS total_compensation
FROM complaints c
JOIN orders o ON c.order_id = o.order_id
GROUP BY o.service_type
ORDER BY complaints DESC;

```

Service type	Complaints	Avg resolution days	Total compensation
Passenger	84	7.74	GBP 1,657.53
Retail	83	7.89	GBP 1,486.14
Parcel	77	8.34	GBP 1,463.65
Business	39	7.95	GBP 810.86
Medical	37	7.57	GBP 740.01

Passenger services generated the most complaints, closely followed by Retail and Parcel. This indicates that customer dissatisfaction is not evenly distributed across service lines and should be addressed with targeted improvement plans.

5. R Analytics and Visualisation

R analytics is used for exploratory summaries and visual communication. The objective is not simply to create charts; it is to present evidence in a form that senior managers can interpret quickly and use to prioritise action.

```

library(ggplot2)
ggplot(deliveries, aes(x = delivery_status)) +
  geom_bar() +
  labs(title = 'Delivery Status Distribution', x = 'Delivery status', y = 'Number of deliveries')

```

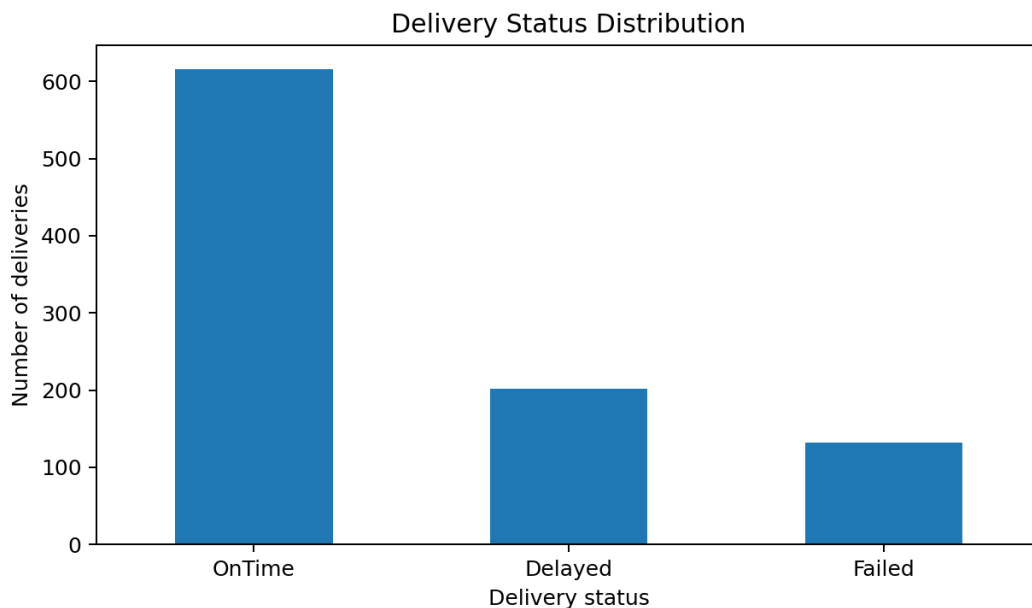


Figure 1. Delivery status distribution from the NorthStar dataset.

Out of 950 delivery records, 616 were on time, 202 were delayed and 132 failed. This confirms that reliability is a measurable operational issue rather than only a narrative problem in the case study.

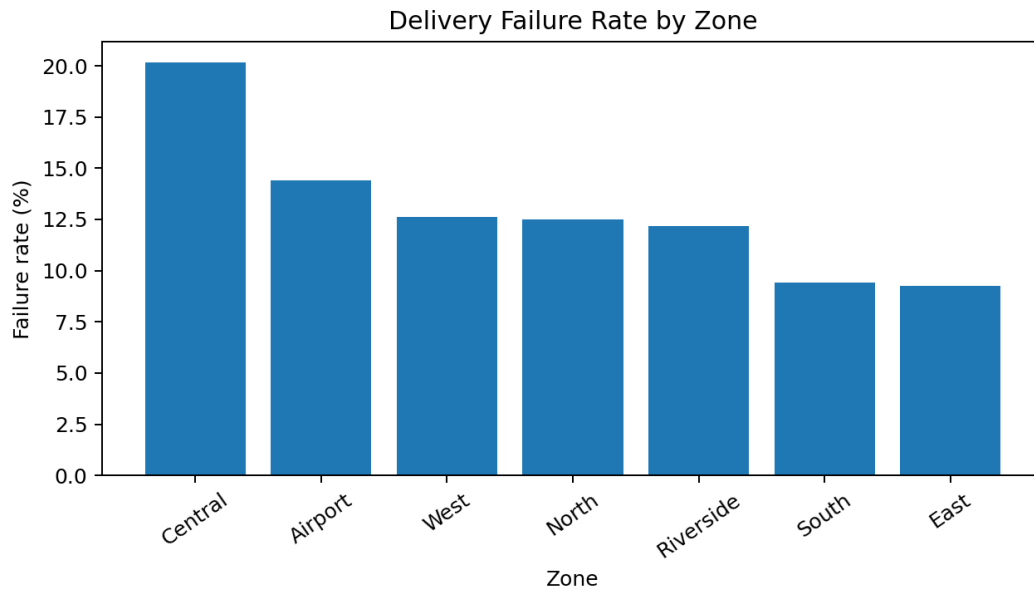


Figure 2. Delivery failure rate by zone.

Central is the weakest zone, while South and East have the lowest failure rates. The pattern suggests that performance problems are concentrated and therefore suitable for targeted operational review.

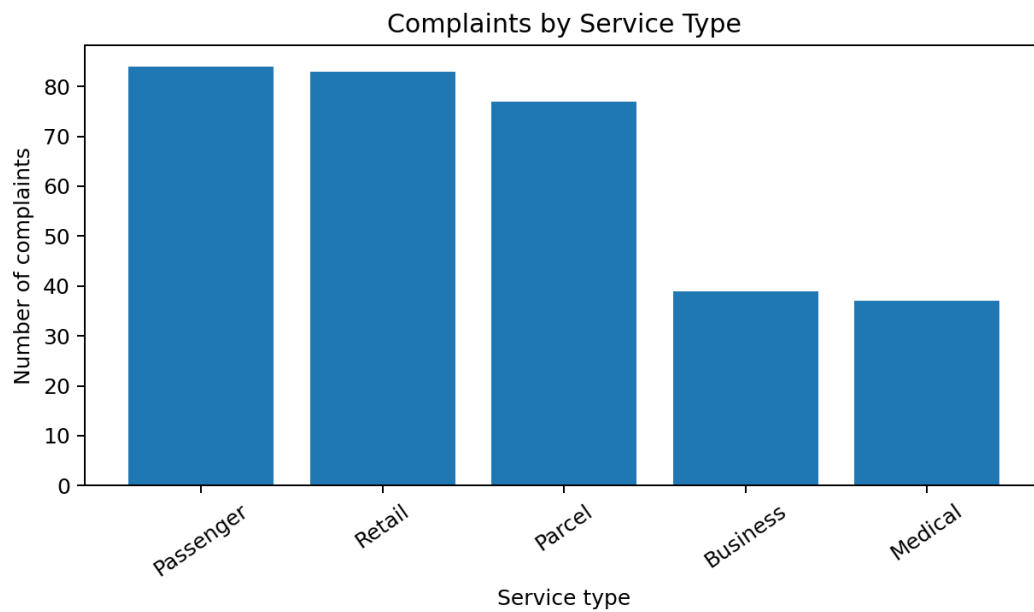


Figure 3. Complaint count by service type.

Passenger services generated 84 complaints. This finding matters because customer-facing services carry reputational risk even when the direct financial compensation amount appears relatively modest.

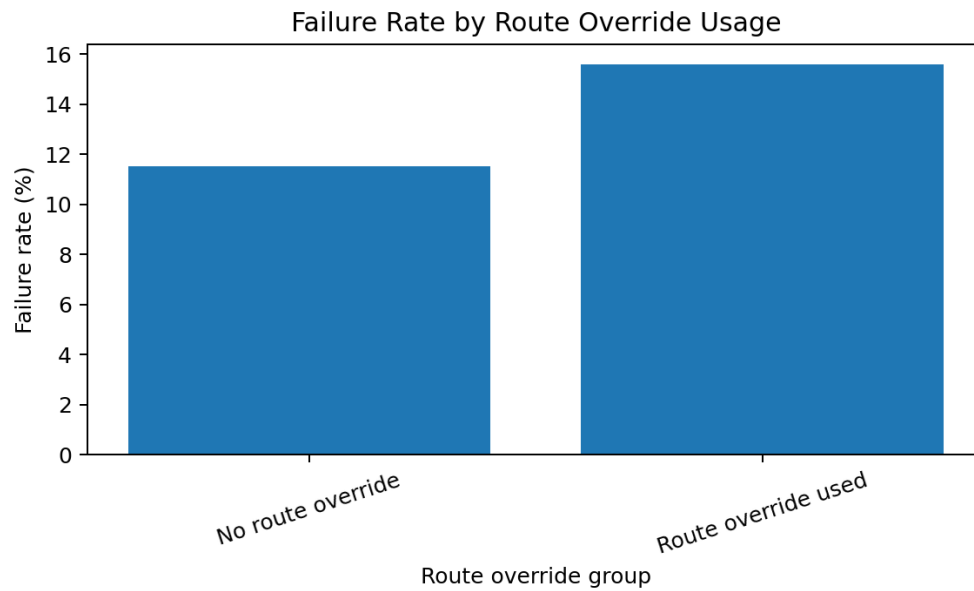


Figure 4. Failure rate by route override usage.

Route override cases had a failure rate of 15.61%, compared with 11.53% for cases without route overrides. This does not prove that overrides cause failures. It does, however, provide a useful operational signal that repeated overrides should be investigated.

6. Python Data Processing and Visualisation

Python was used to clean, join, summarise and export the analysis dataset. Pandas and NumPy are appropriate for this task because they make the multi-file preparation process reproducible and allow risk indicators to be reused in MongoDB service-case documents.

```
deliveries['failed_delivery_flag'] = np.where(deliveries['delivery_status'].str.lower() == 'failed', 1, 0)
deliveries['route_override_flag'] = np.where(deliveries['manual_route_override_count'] > 0, 1, 0)
analysis_df['high_risk_flag'] = np.where(
    (analysis_df['failed_delivery_flag'] == 1) |
    (analysis_df['complaint_count'] > 0) |
    (analysis_df['incident_count'] > 0) |
    (analysis_df['customer_rating_post_delivery'] < 3), 1, 0)
```

Finding	Evidence	Interpretation
High-risk cases are common	610 of 950 delivery cases (64.21%)	More than half of delivery cases require some form of review or monitoring.
Central zone is weakest	20.16% failure rate	Zone-level monitoring should prioritise Central.
Route overrides are riskier	15.61% vs 11.53%	Overrides should be investigated for routing or driver behaviour issues.
InRepair vehicles perform poorly	30.31% failure rate	Vehicle condition should be linked to scheduling and dispatch decisions.
Chat escalation events fail often	50.00% failure rate	Customer-service app workflows need technical review.

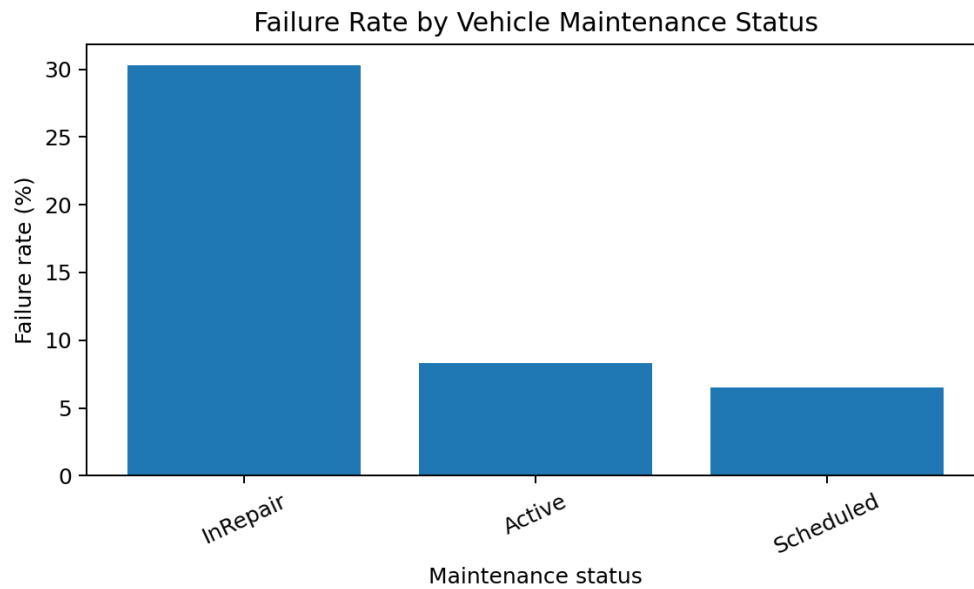


Figure 5. Delivery failure rate by vehicle maintenance status.

InRepair vehicles have a failure rate of 30.31%, far above Active and Scheduled vehicles. This strongly supports the case study concern that vehicle status and route assignments should be analysed together before dispatch decisions are made.

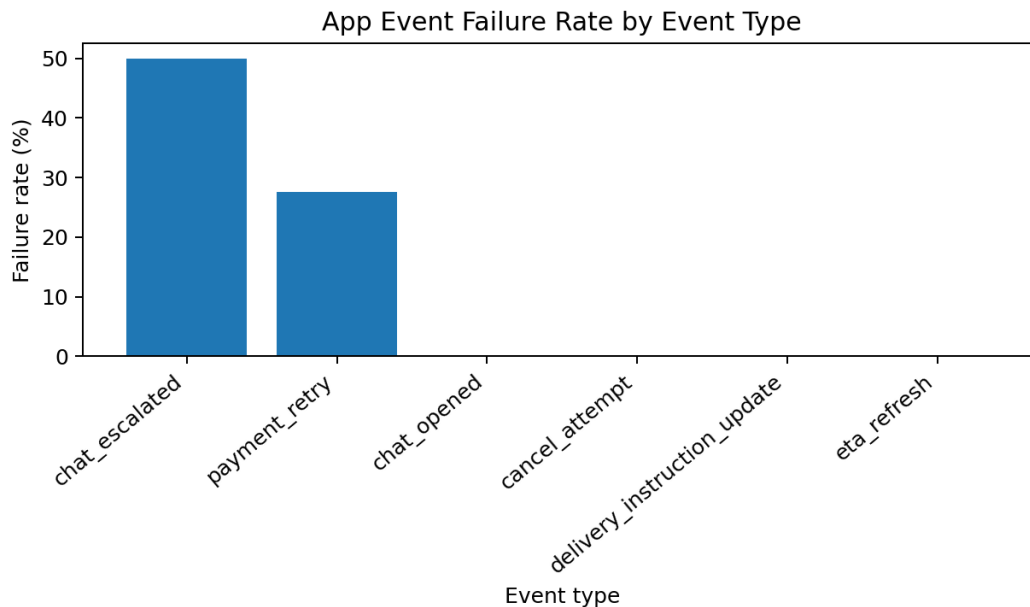


Figure 6. App event failure rate by event type.

The app-event data shows that chat_escalated has a 50.00% failure rate and payment_retry has a 27.54% failure rate. These are not delivery events in the traditional sense, but they can still shape customer satisfaction by affecting communication, resolution and trust.

7. MongoDB Atlas NoSQL Database Design

MongoDB Atlas is recommended for flexible operational records that are nested, event-based or likely to change over time. The strongest design is a `service_cases` collection where each document represents one customer order and embeds the related delivery, complaint, incident and app-event context.

Collection	Purpose	Design decision
<code>service_cases</code>	Integrated operational case view	Embed order, delivery, customer summary, driver summary, vehicle summary, hub, complaints, incidents and app events.
<code>customers</code>	Customer reference data	Store full profile separately because one customer can have many service cases.
<code>drivers</code>	Driver reference data	Store workforce profile separately for reuse across many cases.
<code>vehicles</code>	Fleet reference data	Store vehicle condition and maintenance details.
<code>app_events</code>	Raw platform event records	Keep separate for high-volume analytics while embedding relevant event summaries in service cases.

Example service_cases document

```
{
  'order_id': 'O00938',
  'customer': {'customer_id': 'C0488', 'customer_type': 'SME', 'home_zone': 'North'},
  'order': {'service_type': 'Parcel', 'priority_level': 'High', 'order_value': 72.40},
  'delivery': {'delivery_id': 'DL00001', 'delivery_status': 'Failed',
    'manual_route_override_count': 1, 'customer_rating_post_delivery': 3.07},
  'vehicle': {'vehicle_id': 'V056', 'maintenance_status': 'InRepair'},
  'hub': {'hub_id': 'H05', 'zone': 'Central'},
  'complaints': [],
  'incidents': [],
  'app_events': [{'event_type': 'eta_refresh', 'api_latency_ms': 301, 'success_flag': 1}],
  'risk_flags': {'failed_delivery': true, 'route_override_used': true}
}
```

Embedding is justified because managers usually investigate these records together. A failed delivery review needs the customer, order, hub, driver, vehicle, complaint, incident and app history in one place. A purely relational model would require repeated joins and would be less flexible for evolving event histories.

8. CRUD Operations Using PyMongo

CRUD operations show that MongoDB is not only a modelling exercise. It can support the operational creation, retrieval, updating and archiving of service cases. The GitHub notebook demonstrates these operations using service-case documents and includes PyMongo-style syntax.

```
from pymongo import MongoClient
client = MongoClient('mongodb+srv://USERNAME:PASSWORD_HIDDEN@cluster0.xxxxx.mongodb.net/')
db = client['northstar_mobility']
service_cases = db['service_cases']
```

```

service_cases.insert_one(new_service_case)          # Create
service_cases.find({'risk_flags.failed_delivery': True}) # Read
service_cases.update_one({'order_id': 'O9999'}, {'$set': {'complaints.0.status': 'Resolved'}}) # Update
service_cases.update_one({'order_id': 'O9999'}, {'$set': {'archived': True}}) # Safer delete

```

Operation	PyMongo method	NorthStar use case
Create	insert_one(), insert_many()	Add service cases or bulk load app events
Read	find(), find_one()	Search failed deliveries, complaints or customer history
Update	update_one(), update_many()	Resolve complaints, add incidents or update risk flags
Delete/archive	delete_one() or archived flag	Remove test data or safely archive cases

9. MongoDB Aggregation Queries

MongoDB aggregation pipelines allow NorthStar to analyse nested service-case documents without flattening all event histories into relational tables. The examples below support the core business questions: where failures occur, which services generate complaints and whether route overrides are associated with poorer outcomes.

Aggregation examples

```

pipeline_failed_by_zone = [
    {'$group': {'_id': '$hub.zone', 'total_cases': {'$sum': 1},
               'failed_cases': {'$sum': {'$cond': ['$risk_flags.failed_delivery', 1, 0]}}}},
    {'$addFields': {'failure_rate_percent': {'$round': [{'$multiply': [{'$divide': ['$failed_cases', '$total_cases']}, 100]}, 2]}}}},
    {'$sort': {'failure_rate_percent': -1}}
]

```

```

pipeline_complaints = [
    {'$unwind': '$complaints'},
    {'$group': {'_id': '$order.service_type', 'total_complaints': {'$sum': 1},
               'total_compensation': {'$sum': '$complaints.compensation_amount'}}},
    {'$sort': {'total_complaints': -1}}
]

```

These pipelines are valuable because they keep the operational context intact. For example, a complaint can be analysed together with the service type, delivery status, hub zone, app events and risk flags rather than being treated as a disconnected customer-service record.

10. Indexing and Query Optimisation

Indexing is needed because the case study describes growth in app events, GPS records, complaint histories and service exceptions. Without indexes, MongoDB may perform collection scans, which are inefficient at scale. The coursework dataset is small, so timing differences may be modest, but the design decisions are still important for a realistic operational system.

Index	Reason
-------	--------

order_id	Fast lookup of one service case
customer.customer_id	View all service history for a customer
hub.zone + risk_flags.failed_delivery	Find failed services in a particular zone
order.service_type + risk_flags.has_complaint	Analyse complaint-heavy service lines
delivery.delivery_status + customer_rating_post_delivery	Find failed or low-rated deliveries
app_events.event_type + app_events.success_flag	Investigate failed app workflows

```
service_cases.create_index('order_id')
```

```
service_cases.create_index('customer.customer_id')
```

```
service_cases.create_index([('hub.zone', 1), ('risk_flags.failed_delivery', 1)])
```

```
service_cases.create_index([('order.service_type', 1), ('risk_flags.has_complaint', 1)])
```

```
service_cases.create_index([('delivery.delivery_status', 1), ('delivery.customer_rating_post_delivery', 1)])
```

```
query = {'hub.zone': 'Central', 'risk_flags.failed_delivery': True}
```

```
service_cases.find(query).explain()
```

The expected optimisation improvement is a change from COLLSCAN to IXSCAN, reducing the number of documents examined. Indexes should be used selectively because too many indexes can slow inserts and updates, especially for high-volume app-event data.

11. Key Findings and Business Recommendations

Finding	Evidence from analysis	Recommendation
Fragmented data is the root issue	Service, complaint, incident and app records must be joined to explain failures	Create an integrated service-case data model.
Central is the weakest zone	20.16% failure rate	Review Central hub capacity, routes and vehicle allocation.
Route overrides are riskier	15.61% failure rate with overrides	Monitor overrides and require reason codes for repeated cases.
InRepair vehicles are high risk	30.31% failure rate	Prevent high-risk vehicles from being allocated to priority orders.
Passenger services drive complaints	84 complaints	Prioritise passenger journey reliability and customer communication.
Business services have high failure rate	19.84% failure rate	Review SLAs, pricing and route commitments.
App events reveal digital friction	chat_escalated failure rate 50.00%	Monitor app workflows and API latency as service-quality indicators.

Recommended target architecture

Layer	Technology	Purpose
Structured records	Relational database / SQLite for coursework	Orders, deliveries, drivers, vehicles, hubs and customers
Analytics	SQL in R, R, Python Pandas	Descriptive and diagnostic analysis
Flexible operational	MongoDB Atlas	Complaints, incidents, app events, route

cases		exceptions and service histories
Performance tuning	MongoDB indexes and explain plans	Faster operational and analytical queries
Version control	GitHub and Colab	Transparent reproducible coursework evidence

12. Critical Evaluation and Limitations

The analysis provides useful operational evidence, but it should be interpreted responsibly. It supports diagnostic analytics, not final causal proof. For example, route overrides are associated with a higher failure rate, but the override may be a response to a difficult journey rather than the cause of failure. Similarly, driver-level results should not be used as final performance judgements without considering route difficulty, vehicle allocation, time of day and workload.

A second limitation is scale. The coursework dataset is much smaller than NorthStar's likely real environment. Index performance improvements may therefore be modest in testing, but the indexing strategy remains justified because the case study describes growing volumes of app events, GPS paths, complaint histories and service exceptions.

A future improvement would be predictive modelling. A high-risk delivery model could use zone, service type, priority level, driver rating, vehicle maintenance status, route distance, previous complaints and app latency to predict risk before dispatch. This report does not implement that model because the assignment requirements focus on SQL in R, R analytics, Python processing, MongoDB development and query optimisation.

13. Conclusion

This report has shown that NorthStar's main challenge is the fragmentation of data across operational, customer, driver, vehicle, complaint and digital platform systems. The analysis demonstrates how SQL in R, R visualisation, Python data processing and MongoDB Atlas can be combined to create a stronger decision-making environment.

The findings show clear operational risk: 132 failed deliveries, 202 delayed deliveries, a 20.16% failure rate in Central, a 15.61% failure rate for route override cases and a 30.31% failure rate for vehicles marked InRepair. These results support the case for integrated analytics and better operational control.

The proposed MongoDB service_cases design is suitable because it stores related service histories together, including order, delivery, customer, vehicle, driver, hub, complaints, incidents and app events. Combined with indexing and explain-plan evaluation, it provides a scalable foundation for operational querying and analytics.

14. References

- University of West London. (2026a). Databases and Analytics Assignment Brief 2025-26 S2. School of Computing and Engineering.
- University of West London. (2026b). Case Study: NorthStar Urban Mobility and Logistics. School of Computing and Engineering.

- MongoDB. (n.d.). MongoDB Manual: CRUD Operations, Aggregation Pipelines, Indexes and Query Plans.
- Python Software Foundation. (n.d.). Python Documentation.
- Pandas Development Team. (n.d.). Pandas Documentation.
- R Core Team. (n.d.). R Documentation.
- SQLite. (n.d.). SQLite Documentation.

15. Appendix: GitHub and Colab Evidence

Repository link: <https://github.com/KundanVK18/northstar-database-analytics-assignment>

Evidence item	Location
Notebook 1: Python data processing	https://github.com/KundanVK18/northstar-database-analytics-assignment/blob/main/notebooks/01_python_data_processing.ipynb
Notebook 2: SQL in R and R analytics	https://github.com/KundanVK18/northstar-database-analytics-assignment/blob/main/notebooks/02_sql_in_r_and_r_analytics.ipynb
Notebook 3: MongoDB Atlas and PyMongo	https://github.com/KundanVK18/northstar-database-analytics-assignment/blob/main/notebooks/03_mongodb_atlas_pymongo.ipynb
Outputs folder	https://github.com/KundanVK18/northstar-database-analytics-assignment/tree/main/outputs
Dataset zip	https://github.com/KundanVK18/northstar-database-analytics-assignment/blob/main/data/northstar_dataset.zip
Report folder	https://github.com/KundanVK18/northstar-database-analytics-assignment/tree/main/report